README

Traffic Management System

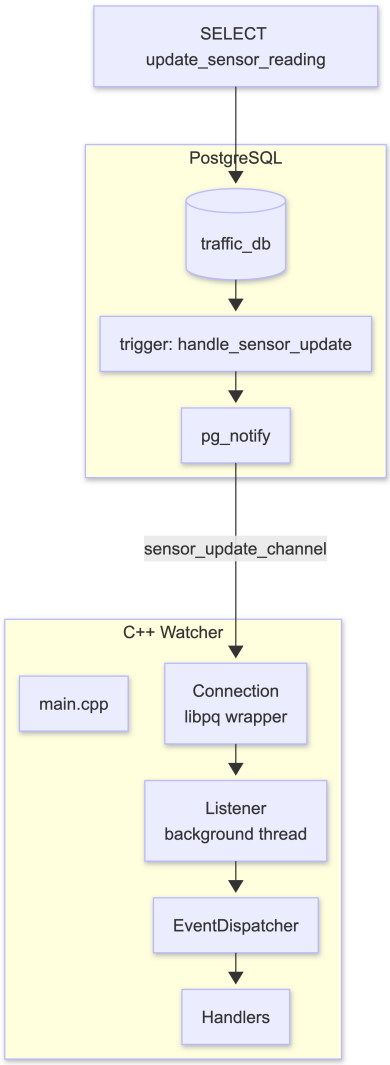
Real-time traffic management monorepo with 3 services:

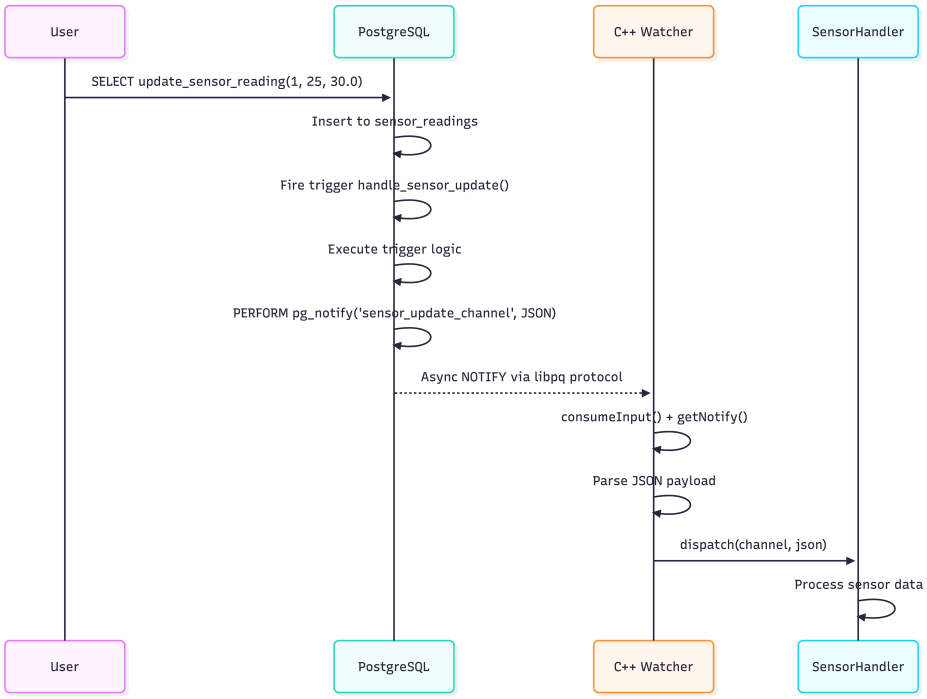
- **C++ Watcher**: PostgreSQL NOTIFY listener → event dispatcher
- **Go Server**: Traffic dashboard API
- **PostgreSQL**: Reactive traffic logic via triggers

Architecture

```
PostgreSQL (Triggers) → NOTIFY → C++ Watcher (Listener → Dispatcher → Handlers → E
                                          ↓
                                   Go Server (REST API)
```

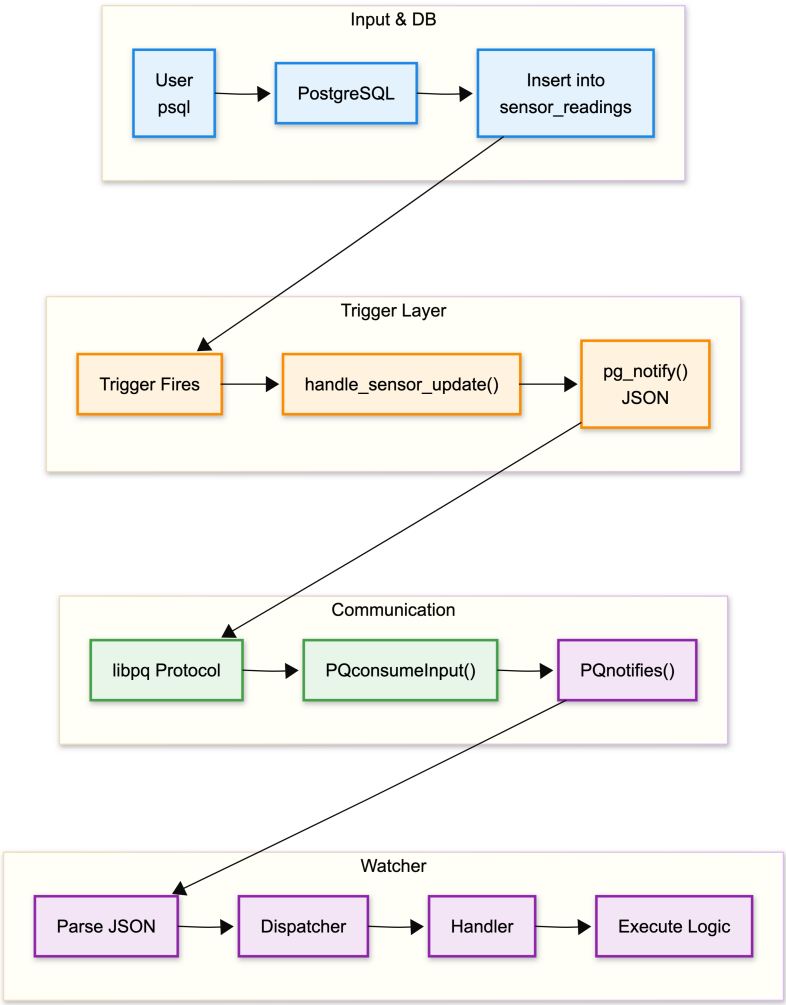
Visual Overview





Untitled 1

```
traffic_db=# \sf handle_sensor_update
CREATE OR REPLACE FUNCTION public.handle_sensor_update()
 RETURNS trigger
 LANGUAGE plpgsql
AS $function$
DECLARE v_approach_id INT;
BEGIN
    SELECT approach_id INTO v_approach_id
    FROM sensors WHERE sensor_id = NEW.sensor_id;

    UPDATE sensors
    SET vehicle_count = NEW.vehicle_count,
        last_updated = NEW.reading_time
    WHERE sensor_id = NEW.sensor_id;

    UPDATE approaches
    SET queue_count = NEW.vehicle_count,
        last_updated = NEW.reading_time
    WHERE approach_id = v_approach_id;

    PERFORM log_traffic_history(v_approach_id, NEW.vehicle_count, NEW.reading_time);
    PERFORM recompute_phase_for_approach(v_approach_id);

    PERFORM pg_notify(
        'sensor_update_channel',
        json_build_object(
            'type', 'SENSOR_UPDATE',
            'sensor_id', NEW.sensor_id,
            'vehicle_count', NEW.vehicle_count,
            'avg_speed', NEW.avg_speed,
            'reading_time', NEW.reading_time,
            'approach_id', v_approach_id
        )::text
    );

    RETURN NEW;
END;
$function$
traffic_db=#
```



Project Structure

```
./
├── README.md              # This file
├── AGENTS.md              # Project knowledge base
├── docs/                  # Documentation (LaTeX reports, diagrams)
│   └── TrafficManagementSystem.tex
├── watcher/             # C++ event listener (CMake + C++20)
│   ├── CMakeLists.txt    # Portable build (auto-detects PostgreSQL)
│   ├── README.md         # Detailed watcher documentation
│   ├── include/watcher/
│   │   ├── config.hpp
│   │   ├── connection.hpp    # libpq wrapper (RAII)
│   │   ├── dispatcher.hpp    # Event routing + rule passing
│   │   ├── listener.hpp      # Background NOTIFY polling
│   │   └── rules/
│   │       └── rule_loader.hpp # ECA XML parser (pugixml)
│   ├── src/
│   │   ├── main.cpp            # Entry point
│   │   ├── config.cpp
│   │   ├── connection.cpp
│   │   ├── listener.cpp
│   │   ├── dispatcher.cpp
│   │   ├── rules/
│   │   │   └── rule_loader.cpp
│   │   └── handlers/
```

```
│   │       ├── sensor_handler.cpp   # Executes rules → SQL
│   │       └── signal_handler.cpp  # Executes rules → SQL
│   ├── build/              # (gitignored)
│   └── include/images/   # Architecture diagrams
│       ├── archi.png
│       ├── swimlane.png
│       ├── cleaner.png
│       └── flow.png
├── server/               # Go API server
│   └── main.go
└── sql/                 # PostgreSQL schema + triggers
    ├── db_create.sql      # Schema + triggers + functions
    ├── db_data.sql        # Seed data
    ├── db_test.sql        # Test queries
    ├── db_watcher_test.sql # Watcher integration test
    └── xml/
        ├── eca-rules.xsd       # ECA rules schema
        └── eca-rules-example.xml # Example rules (9 rules)
```

Quick Start

1. Prerequisites

- PostgreSQL 17+ running
- C++20 compiler (Clang/GCC)
- Go 1.25+
- CMake 3.16+

2. Database Setup

```
createdb traffic_db
psql -U dheerajmurthy -d traffic_db -f sql/db_create.sql
psql -U dheerajmurthy -d traffic_db -f sql/db_data.sql
```

3. Build Watcher

```
cd watcher
# Clean any previous failed builds
rm -rf build/* _deps

# Build (portable - auto-detects PostgreSQL)
cd build
cmake .. && make -j4

# Run
./watcher
```

Expected output:

```
Connecting to traffic_db...
Connected.
Loaded 9 rules (version 1.0)
  - [rule-001] High Traffic Response (priority 10)
  - [rule-002] Low Traffic Optimization (priority 5)
  - [rule-003] Rush Hour Corridor (priority 20)
  ...
Listening for events... (Ctrl+C to exit)
```

4. Run Go API Server

```
cd server && go run main.go
# Server starts on :8080
```

5. Test Events

```
# In another terminal - trigger sample events
psql -U dheerajmurthy -d traffic_db -f sql/db_watcher_test.sql
```

Watcher output:

```
[sensor_update_channel] {"type":"SENSOR_UPDATE","sensor_id":1,...}
Processing sensor update...
  Data: { "sensor_id": 1, "vehicle_count": 50, ... }
  Matching rules: 3
    Rule: High Traffic Response (priority 10)
      ACTION: Set phase 1 green=90
    Rule: Low Traffic Optimization (priority 5)
      ACTION: Adjust green time 10s (min=10 max=120)
```

Components

C++ Watcher

Purpose: Listen for PostgreSQL NOTIFY events and execute ECA rules.

Key Features:

- **RAII Connection**: `unique_ptr<PGconn, PGconnDeleter>` auto-cleans
- **Background Listener**: Separate thread polls `PQconsumeInput()`
- **Event Dispatcher**: Routes by channel name to handlers

- **ECA Rule Engine**: XML-based rules parsed with pugixml (header-only)
- **Automatic Actions**: Rules match → SQL executed against DB

Dependencies:

- PostgreSQL (libpq) - auto-detected via CMake
- nlohmann/json v3.11.3 - via FetchContent
- pugixml v1.14 - via FetchContent (header-only)

Go Server

Purpose: REST API for traffic dashboard.

Endpoints (to be implemented):

- `GET /api/sensors` - List sensor data
- `GET /api/junctions` - Junction status
- `POST /api/signal` - Manual signal control

PostgreSQL

Purpose: Reactive traffic logic via triggers.

Event Channels: | Channel | Event Type | |---------|----------| | `sensor_update_channel` | Vehicle count, speed data | | `signal_update_channel` | Phase changes, timing | | `junction_event_channel` | Junction-specific events |

Key Functions:

- `handle_sensor_update()` - Trigger on sensor inserts
- `recompute_phase_for_approach()` - Recalculates traffic phase
- Sends NOTIFY with JSON payload

ECA Rules

Rules loaded from `sql/xml/eca-rules-example.xml` (9 rules included).

Example Rule

```
<rule id="rule-001" priority="10" enabled="true">
  <name>High Traffic Response</name>
  <description>When queue exceeds 40, extend green time</description>
  <event>
    <sensor-update vehicle-count-min="41"/>
  </event>
  <condition>
```

```
    <queue-threshold threshold="40" comparison="gt"/>
  </condition>
  <action>
    <set-phase-timing phase-id="1" green-time="90" is-active="true"/>
  </action>
</rule>
```

Supported Actions

Action	SQL Executed
`set-phase-timing`	`UPDATE phases SET green_time=X`
`set-active-phase`	`UPDATE approaches SET green_time=X`
`enable-corridor-free`	`UPDATE corridors SET mode='CORRIDOR_FREE'`
`reset-to-dynamic`	`UPDATE junctions SET mode='DYNAMIC'`
`set-mode`	`UPDATE junctions SET mode='...'`
`adjust-green-time`	`UPDATE approaches SET green_time=green_time+Y`

Documentation

- **Watcer Details**: `watcher/README.md`
- **LaTeX Report**: `docs/TrafficManagementSystem.tex` (import to Overleaf)
- **ECA Rules Spec**: `sql/report/xml_input_spec_fixed.md`
- **SRSS Report**: `sql/report/srs.md`

Build System

CMake auto-detects PostgreSQL:

- macOS: Homebrew (`/opt/homebrew/opt/postgresql@17`)
- Linux: System paths
- Custom: Set `PostgreSQL_ROOT` or `WATCHER_ROOT` env var

Portable - no hardcoded paths. Works on Apple Silicon + Intel + Linux.

Running the Full System

```
# Terminal 1: Start PostgreSQL (if not running)
pg_ctl -D /usr/local/var/postgres start

# Terminal 2: Start watcher
```

```
cd watcher/build && ./watcher

# Terminal 3: Start Go server
cd server && go run main.go

# Terminal 4: Trigger test events
psql -d traffic_db -f sql/db_watcher_test.sql
```

Future Improvements

- [ ] Connect handlers to ECA rules for automatic response
- [ ] Add time-window aggregation for sensor data
- [ ] Implement adaptive signal timing algorithms
- [ ] Add Prometheus metrics endpoint
- [ ] WebSocket API for real-time dashboard
- [ ] Unit tests for handlers
- [ ] Complete Go REST API implementation

Technologies Used

- **PostgreSQL 17**: NOTIFY, triggers, stored functions
- **C++20**: Watcher with pugixml, nlohmann-json
- **Go 1.25**: REST API server
- **CMake**: Build system (portable)
- **pugixml v1.14**: Header-only XML parser
- **nlohmann/json v3.11.3**: JSON parsing

License

This project is licensed under the MIT License - see the ./LICENSE.md file for details.

Author: Dheeraj Murthy

Date: April 2026